DSA03-NATURAL LANGUAGE PROCESSING ASSIGNMENT-1

ASSIGNMENT 1 – CO1

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Text Normalisation and Search for a Hospital Record System

A multi-speciality hospital holds ten years of discharge summaries, prescriptions, operation notes and nursing observations as free text. Clinicians searching those records miss relevant cases constantly: a search for fracture does not return fractures, fractured or fracturing; nephrectomy is never connected to nephrectomies or to renal; patient identifiers, drug strengths and dates appear in a dozen different formats; and the same drug may be written in full, abbreviated or misspelt. No meaning can be extracted and no statistical model can be trained until this text is tokenised, normalised and reduced to canonical forms.

You are the language-engineering team. Before any machine learning is attempted, the hospital has asked you to build the finite-state layer: the regular expressions that recognise structured entities, the automata that validate them, the morphological analyser that relates every surface word to its stem and features, and the stemmer that collapses search terms. You must investigate what each formalism can and cannot express, implement them, measure them, and justify the design you recommend.

Planning data and constraints

- Corpus: at least 500 de-identified clinical records (real or synthetic, but documented), totalling at least 25,000 word tokens and covering at least four record types.
- Pattern inventory: at least 12 structured entity types to be recognised by regular expression — patient identifier, admission and discharge date, drug name with

strength, dosage frequency, laboratory value with unit, blood pressure, temperature, ICD code, ward and bed, contact number, age, and duration of symptom.

- Automata: at least four finite-state automata, each with at least six states, including one entity format that cannot be recognised conveniently without non-determinism.
- Morphological resources (build and document your own): a lexicon of at least 300 stems, at least 20 inflectional affixes and at least 15 derivational affixes, with irregular forms listed separately.
- Evaluation data: at least 40 clinician queries with relevance judgements over the corpus, and at least 200 word forms hand-annotated with their stem and features.
- Any manual effort (pattern writing, lexicon building, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The system must now ingest notes captured by optical character recognition from scanned handwritten forms, and notes typed in Romanised regional language mixed with English. Re-evaluate your original patterns, automata and analyser and decide whether to retain, modify or redesign them.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis 3 Marks

Guiding Enquiry

Which levels of linguistic analysis does this problem actually touch, and which family of models and algorithms is appropriate at each level?

Investigate and interpret:

- Describe the levels of linguistic analysis — phonology, morphology, syntax, semantics, pragmatics and discourse — and state, with an example drawn from your own corpus, which levels the record-search problem requires and which it can safely ignore.
- Distinguish the three model families used throughout this unit: state machines, formal rule systems and probabilistic models. Name one task in the hospital problem that each family suits best and say why the other two would be the wrong choice for it.
- Explain why ambiguity is the central problem of natural language processing, and identify at least four distinct ambiguities present in your corpus — lexical, morphological, segmentation and abbreviation ambiguity.
- Identify where search appears in this unit (state-space search in automata, backtracking in matching) and state the resources each consumes: time, memory, annotated data and expert effort.
- Set out, as a labelled diagram, the pipeline you will build in the remaining tasks, naming the input, the output and the formalism of every stage.

Evidence Expected

A reasoned problem framing that maps the hospital task onto levels of analysis, model families and a concrete, staged pipeline.

TASK 2 – Basic Regular Expression Patterns 4 Marks

Guiding Enquiry

What can the basic regular expression operators express, and how are they combined into patterns that match real clinical strings exactly?

Investigate and interpret:

- Define the basic operators — concatenation, disjunction, Kleene star and plus, optionality, character classes, ranges, negated classes, anchors and counters — and give one minimal clinical example of each.
- Write and document a regular expression for each of the twelve entity types in your pattern inventory, stating for every pattern both what it must match and what it must not.
- Demonstrate greedy and non-greedy matching on a clinical sentence containing two dosage expressions, and explain precisely why the two quantifiers return different spans.
- Explain operator precedence and grouping, and show with a worked example how a misplaced disjunction silently changes the language the expression denotes.
- Evaluate every pattern over the corpus, reporting true positives, false positives and false negatives, and name the specific linguistic cause of each error class.

Evidence Expected

A documented pattern inventory with precise match specifications and a per-pattern error analysis carried out on real data.

TASK 3 – Regular Expressions for Tokenisation and Normalisation 4 Marks

Guiding Enquiry

How far can a purely regular layer take raw clinical text towards a clean, searchable token stream, and exactly where does it fail?

Investigate and interpret:

- Specify your tokenizer as a single regular expression, defining precisely what counts as a token — hyphenated terms, abbreviations containing full stops, decimal values, units written against numbers — and justify each decision.
- Apply maximal-munch (longest-match) tokenisation and show on at least three examples how the result differs from a naive split on whitespace.
- Build substitution rules that fold case, expand at least thirty clinical abbreviations and canonicalise date and unit formats; show the complete before-and-after text of one full record.
- Identify at least three normalisation decisions that are irreversible because information is destroyed, and argue when that loss is acceptable and when it is not.
- Give one requirement of the hospital task that no regular expression can express, and argue informally why — for example, matching balanced parentheses in a nested clinical note.

Evidence Expected

A working regular-expression tokenizer and normaliser with worked examples and a clear statement of the limits of regularity.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence 5 Marks

Guiding Enquiry

How do automata recognise exactly the same languages as regular expressions, and what does removing non-determinism cost?

Investigate and interpret:

- Define a finite automaton formally as the five-tuple $(Q, \Sigma, \delta, q_0, F)$, then design a deterministic automaton for at least four validators from your pattern inventory, giving both the transition diagram and the transition table for each.
- Design a non-deterministic automaton with ε-transitions for one entity whose format has optional parts, and trace its behaviour on one accepted and one rejected string, showing the set of states live at every step.
- Convert that NFA to a DFA by subset construction, showing the complete construction table, then minimise the DFA and report the state counts before and after.
- Apply Thompson's construction to turn one Task 2 regular expression into an automaton, and derive a regular expression back from one of your automata, demonstrating the equivalence in both directions.
- State the pumping lemma for regular languages and use it to prove that one specific clinical pattern is not regular — for example a note requiring equal numbers of opening and closing brackets — and relate the proof to your finding in Task 3.

Evidence Expected

Correct automata with full subset-construction and minimisation tables, a demonstrated two-way equivalence with regular expressions, and a valid non-regularity proof.

TASK 5 – Inflectional Morphology 3 Marks

Guiding Enquiry

Which inflectional processes appear in clinical English, and how are surface forms systematically related to their stems?

Investigate and interpret:

- Define morpheme, stem, affix, inflection and paradigm, and separate inflection from derivation using the standard criteria of category preservation, productivity and position relative to other affixes.
- List every inflectional paradigm present in your corpus — noun number, verb tense and aspect, subject agreement, adjective comparison — and give the full paradigm for at least five clinical lexemes.
- State the orthographic rules governing English inflection — e-deletion, consonant doubling, y-to-i, the sibilant -es rule — precisely enough to be implemented, and give for each one a counter-example on which it over-generates.
- Identify the irregular forms in your corpus and explain why each must be listed in the lexicon rather than derived by rule.
- Measure the effect of inflection: report the type-to-lemma ratio of your corpus and the proportion of failures in your query set that inflection alone explains.

Evidence Expected

Precisely stated inflectional rules with paradigms, an irregular list, and a quantified measurement of the effect of inflection on retrieval.

TASK 6 – Derivational Morphology 4 Marks

Guiding Enquiry

How does derivation build the specialised clinical vocabulary, and why is it harder to handle than inflection?

Investigate and interpret:

- Define derivation and list at least fifteen derivational affixes attested in your corpus — for example -itis, -ectomy, -ology, -osis, -aemia, -ation, -able, -ive, nephro-, hyper-, hypo-, re-, un- — stating the category change each one causes.
- Build the derivational family of at least six clinical roots, showing the order in which affixes attach and identifying at least two orders that are impossible.
- Explain semantic non-compositionality with clinical examples in which the meaning of the derived word cannot be predicted from its parts, and state what this implies for a search system that stems aggressively.
- Compare the productivity of your affixes empirically: count types and tokens for each in the corpus and classify each affix as productive or fossilised, with the evidence for the classification.
- Decide whether derivational relatives should be conflated for search, citing at least two cases where conflation helps the clinician and two where it produces a clinically misleading match.

Evidence Expected

A documented derivational analysis with an affix inventory, productivity measurements and a reasoned position on conflation.

TASK 7 – Finite-State Morphological Parsing 5 Marks

Guiding Enquiry

How can a transducer relate a surface form to its stem and features, and run in both directions on the same machine?

Investigate and interpret:

- Define a finite-state transducer as a two-tape automaton and explain the lexical and surface tapes with one fully worked clinical example.
- Build your lexicon as an FST organised into continuation classes, covering at least three parts of speech together with the inflectional and derivational affixes of Tasks 5 and 6.
- Hand-trace the parsing of at least five surface forms, showing the state sequence, the symbols consumed and emitted on each tape, and the final analysis — for example fractures → fracture +N +PL.
- Run the same transducer in the generation direction, producing surface forms from stem-plus-feature input, and show explicitly that parsing and generation use one machine and not two.
- Handle ambiguity: give at least two surface forms with more than one analysis, show the transducer returning all of them, and state how a downstream component would choose between them.

Evidence Expected

A working transducer specification with traced analyses in both directions and an explicit treatment of ambiguity.

TASK 8 – Orthographic Rules and Two-Level Morphology 4 Marks

Guiding Enquiry

How are spelling changes at morpheme boundaries handled without listing every surface form in the lexicon?

Investigate and interpret:

- Explain why a lexicon transducer alone cannot produce correct surface spellings, using at least three examples in which plain concatenation yields the wrong string.
- Write at least five orthographic rules in the standard rewrite notation (a → b / c ___d), stating the context of each rule precisely.
- Implement each rule as a transducer and compose the rule transducers with the lexicon FST; draw the cascade and state the order dependencies between the rules.
- Contrast this sequential cascade with the parallel two-level (Koskenniemi) formulation, and explain the practical advantage each has over the other.
- Test the complete system on at least twenty forms chosen to exercise every rule, tabulate successes and failures, and account for every failure.

Evidence Expected

Correct rule transducers, a documented composition order, and a tested cascade with full error accounting.

TASK 9 – The Porter Stemmer 5 Marks

Guiding Enquiry

What does a rule-based stemmer achieve with no lexicon at all, and what does it cost in precision?

Investigate and interpret:

- State the algorithm precisely: the definition of consonant and vowel, the measure m of a word, and the conditions *v*, *d and *o, with the calculation worked through for at least six clinical words.
- Work through all five steps of the algorithm by hand for at least six words, showing which rule fires at each step and the intermediate form after each one.
- Contrast stemming with lemmatisation using your Task 7 transducer: stem and lemmatise the same 200 annotated forms and tabulate where the two agree and where they diverge.
- Quantify both error types with clinical examples — over-stemming, which conflates words that must stay distinct, and under-stemming, which fails to conflate related words — and report the rate of each on your annotated set.
- Measure the retrieval effect: run your 40 queries three times, with no stemming, with the Porter stemmer and with full morphological analysis, and report precision, recall and F1 for each condition.

Evidence Expected

Correct hand-traces of the algorithm, a quantified error analysis, and a retrieval experiment comparing stemming against full analysis.

TASK 10 – Implement and Test the Pipeline 3 Marks

Guiding Enquiry

Can a working implementation reproduce the behaviour you derived by hand?

Investigate and interpret:

- Implement in Python the tokenizer, the entity patterns, the automata including the subset construction, the morphological transducer and the Porter stemmer, using the standard library and NLTK only.
- Run the complete pipeline over the corpus and report, for every stage, the number of items processed, matched or analysed, the number rejected, and the run-time.
- Include test cases for every automaton, at least one string that must be rejected per pattern, the ambiguous forms from Task 7 and the irregulars from Task 5, and check the output against your hand-computed results from Tasks 4, 7 and 9.
- Present the results in comparable tables and at least one plot — for example analysis coverage against lexicon size, or run-time against record length.

Evidence Expected

Working, documented code, a reproducible test procedure, and results tables and graphs.

TASK 11 – Compare, Recommend and Defend the Normalisation Design 2 Marks

Guiding Enquiry

Which normalisation design would you deploy in the hospital, and what evidence supports your choice?

Investigate and interpret:

- Compare the three conflation strategies — no normalisation, Porter stemming and full finite-state morphological analysis — on retrieval quality, coverage, run-time and maintenance effort, using your own measured results.
- Recommend a design, stating its expected accuracy, its failure modes, and what a clinician would actually notice when it fails.
- Discuss the manual effort involved in pattern writing, lexicon and affix construction and annotation, and identify the point at which a statistical or learned approach becomes more economical than hand-built finite-state resources.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the highest recall alone is not sufficient.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The system must now ingest two new sources. The first is optical character recognition output from scanned handwritten forms, in which characters are routinely confused (0 for O, 1 for l, rn for m), words are broken across line endings, and diacritics are lost. The second is notes typed by staff in Romanised regional language mixed with English, for example “patient ku fever irundhuchu, paracetamol 500mg kuduthen”. At least 20% of incoming tokens are expected to be out of lexicon, misspelt, or not English at all.

Re-evaluate your original design under the changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- Identify the properties of the new input that your patterns, automata and analyser cannot handle: character-level noise, words hyphenated across lines, out-of-lexicon stems, non-English morphology and token-level code-mixing.
- Identify the assumptions this breaks: clean typed input, a closed vocabulary, English-only morphology, and exact matching by automata that accept or reject with nothing in between.
- Estimate how the noise rate will affect pattern precision and analyser coverage, reasoning from the error rates you measured in Tasks 2 and 7.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

B. Evaluation of the Original Design – 2 Marks

Guiding Enquiry

How would your recommended pipeline actually behave on the new input?

- Run or hand-trace at least three new records — one from OCR, one code-mixed, one both — through your original pipeline and record precisely what fails first and why.
- Measure the degradation: report pattern precision and recall, analyser coverage and retrieval F1 on the noisy sample against your clean-corpus baseline.
- Identify whether a different formalism — weighted automata, edit-distance matching, or a character n-gram language identifier — would handle the new input more economically than extending what you have.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

C. Adaptation or Redesign – 2 Marks

Guiding Enquiry

Should the finite-state layer be extended, replaced or supplemented?

- Choose retain, modify or redesign — for example placing a weighted edit-distance transducer in front of the analyser, extending the lexicon with regional stems, adding

token-level language identification, or falling back to stemming for out-of-lexicon tokens.

- Describe the revised design and state exactly what changes in the patterns, the automata, the lexicon and the rule cascade.
- Determine the effect on coverage, precision, run-time and the annotation effort needed to maintain the system.

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and a quantified effect.

D. Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one, given the evidence you generated?

- Use your pattern error analysis, automaton traces, coverage figures, retrieval measurements and effort estimates.
- Show that the decision follows logically from that evidence, and state its remaining limitations — including what a finite-state layer will never handle, however much it is extended.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

ANSWER:

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis

1.1 Introduction

Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to process, understand, and analyse human language.

Hospitals store large amounts of information in discharge summaries, prescriptions, operation notes, and nursing observations. These records contain medical terms, abbreviations, dates, drug names, and different word forms.

A search for the word "fracture" may fail to retrieve records containing "fractures", "fractured", or "fracturing".

The proposed system uses text normalisation and morphological analysis to identify related word forms and improve clinical information retrieval.

1.2 Levels of Linguistic Analysis

Level	Description	Hospital example
Phonology	Study of speech sounds	Identifying spoken medical terms
Morphology	Study of word formation	Fracture, fractures, fractured
Syntax	Study of sentence structure	The patient received medication
Semantics	Study of meaning	Understanding that nephrectomy involves removal of a kidney
Pragmatics	Meaning based on context	Understanding a doctor's instruction
Discourse	Relationship between sentences	Connecting symptoms and diagnosis across a report

The hospital search system mainly requires morphology, basic syntax, and some semantic information.

Morphology is necessary because related medical terms occur in different forms. Syntax helps identify structured expressions such as dosage instructions. Semantics helps distinguish clinically related terms.

Phonology, pragmatics, and discourse analysis are not essential for the initial text normalisation layer, although they may become useful in future systems.

1.3 Model Families

Three important model families are state machines, formal rule systems, and probabilistic models.

State machines: These represent a system using states and transitions. They are suitable for validating patient identifiers, dates, and other structured entities.

Formal rule systems: These use predefined linguistic rules to transform words. They are suitable for identifying stems, suffixes, and morphological features.

Probabilistic models: These use probabilities learned from data. They are suitable for handling ambiguous words, noisy text, and uncertain language patterns.

For example, a finite automaton is suitable for validating a patient identifier because the format follows a fixed structure. A probabilistic model would be unnecessarily complex for this task.

Similarly, a morphological rule system is suitable for analysing regular forms such as fractures. A state machine alone cannot determine the complete linguistic meaning of every word.

1.4 Ambiguity in Natural Language

Ambiguity occurs when a word or expression has more than one possible interpretation.

The major types of ambiguity in hospital records are:

1. **Lexical ambiguity:** A word has multiple meanings. For example, "cold" may refer to a low temperature or a common illness.
2. **Morphological ambiguity:** A word may have multiple morphological analyses.
3. **Segmentation ambiguity:** A sequence of characters may be divided into words in different ways.
4. **Abbreviation ambiguity:** The same abbreviation may represent different medical terms.

Ambiguity makes it difficult to identify the correct meaning of clinical records.

1.5 Search and Resource Requirements

Search appears in the following stages:

- State-space search is used while processing automata.
- Backtracking may occur during regular expression matching.
- Dictionary lookup is used during morphological analysis.
- Search algorithms retrieve relevant hospital records.

The resources required include processing time, memory, annotated data, and expert effort.

Regular expressions require limited linguistic resources but may become expensive when poorly designed. Morphological analysis requires a carefully constructed lexicon and linguistic rules.

1.6 Proposed Pipeline

Hospital Records

|

v

Text Preprocessing

|

v

Regular Expression Tokenisation

|

v

Entity Recognition

|

v

Text Normalisation

|

v

Finite-State Morphological Analysis

|

v

Porter Stemming

|

v

Search Index

|

v

Relevant Hospital Records

Each stage contributes to the transformation of raw clinical text into a searchable representation.

Conclusion: The proposed system combines regular expressions, automata, and morphological analysis to improve clinical record retrieval.

TASK 2 – Basic Regular Expression Patterns

2.1 Introduction

A regular expression (Regex) is a sequence of characters used to identify patterns in text.

Regular expressions are useful for identifying structured information such as dates, patient identifiers, drug strengths, and laboratory values.

2.2 Basic Regular Expression Operators

Operator	Meaning	Example
Concatenation	Joins patterns	AB
Disjunction	Matches either pattern	A|B
Kleene star	Zero or more repetitions	A*
Plus	One or more repetitions	A+
Optionality	Zero or one occurrence	A?
Character class	Matches one character from a set	[ABC]
Range	Matches characters within a range	[0-9]
Negated class	Matches characters outside a set	[^0-9]
Anchors	Match the beginning or end	^abc$
Counters	Specify repetition count	A{2,4}

2.3 Clinical Entity Patterns

The following expressions identify 12 structured entity types.

The examples use Python regular expression syntax.

```
import re

patterns = {
  "Patient ID": r"\bP\d{6}\b",

  "Admission Date": r"\b\d{4}-\d{2}-\d{2}\b",

  "Discharge Date": r"\b\d{4}-\d{2}-\d{2}\b",

  "Drug Strength": r"\b[A-Za-z][A-Za-z-]*\s+\d+(?:\.\d+)?\s*(?:mg|g|mcg|ml)\b",

  "Dosage Frequency": r"\b(?:once|twice|thrice|\d+\s*(?:times|x)\s*(?:daily|a day)|OD|BD|TDS|QID)\b",

  "Laboratory Value": r"\b\d+(?:\.\d+)?\s*(?:mg/dL|g/dL|mmol/L|U/L|%|mmHg)\b",
```

```
    "Blood Pressure": r"\b\d{2,3}/\d{2,3}\s*mmHg\b",

    "Temperature": r"\b\d{2}(?:\.\d+)?\s*(?:°C|C|°F|F)\b",

    "ICD Code": r"\b[A-Z]\d{2}(?:\.\d{1,4})?\b",

    "Ward and Bed": r"\bWard\s+[A-Z0-9-]+\s*,?\s*Bed\s+\d+\b",

    "Contact Number": r"\b(?:\+91[- ]?)?[6-9]\d{9}\b",

    "Age": r"\b\d{1,3}\s*(?:years?|yrs?)\b",

    "Symptom Duration": r"\b\d+\s*(?:days?|weeks?|months?|years?)\b"
}
```

These expressions identify common clinical patterns. They must be validated against the actual corpus before deployment.

2.4 Matching and Non-Matching Examples

Entity	Matching example	Non-matching example
Patient ID	P123456	P12345
Date	2026-11-05	2026-13-40
Drug strength	Paracetamol 500mg	Paracetamol
Dosage frequency	2 times daily	2 daily
Laboratory value	120 mg/dL	120
Blood pressure	120/80 mmHg	120/80
Temperature	37°C	37 degrees
ICD code	A12.3	123.4
Ward and bed	Ward A1, Bed 12	Ward A1
Contact number	9876543210	12345
Age	45 years	45
Symptom duration	3 days	3 hours

The date expression recognises the correct format but does not validate whether the date exists. A production implementation must include additional date validation.

2.5 Greedy and Non-Greedy Matching

Consider the sentence:

"Patient received 500mg paracetamol and 250mg ibuprofen."

A greedy expression:

r"\d+mg.*"

may match:

500mg paracetamol and 250mg ibuprofen

A non-greedy expression:

r"\d+mg.*?"

matches the shortest possible portion after the dosage.

Greedy quantifiers attempt to match as much text as possible. Non-greedy quantifiers attempt to match as little text as possible while satisfying the expression.

2.6 Operator Precedence

Consider:

ab|cd

This expression matches either ab or cd.

However:

a(b|c)d

matches abd or acd.

Parentheses control the grouping of alternatives.

For example:

fracture|fractures

matches either word.

fracture(s)?

matches both forms using an optional suffix.

2.7 Evaluation

The pattern inventory must be evaluated using:

- True positives
- False positives
- False negatives

A true positive is a correctly recognised entity.

A false positive is a string incorrectly recognised as an entity.

A false negative is an entity that the pattern fails to recognise.

For example, an expression designed to recognise drug strengths may fail when a drug name contains multiple words.

Conclusion: Regular expressions provide an efficient method for recognising structured clinical information, but their accuracy depends on the quality of the patterns and the variation present in the corpus.

TASK 3 – Regular Expressions for Tokenisation and Normalisation

3.1 Introduction

Tokenisation divides text into smaller units called tokens.

Normalisation transforms different representations of the same information into a standard form.

For example:

Patient has fractures.

The tokens are:

Patient

has

fractures

.

3.2 Tokenisation Pattern

```
import re

token_pattern = re.compile(
    r"\d{4}-\d{2}-\d{2}|"
    r"[A-Za-z]+(?:[-'][A-Za-z]+)*|"
    r"\d+(?:\.\d+)?|"
    r"[^\w\s]"
)

text = "Patient has 120.5 mg/dL glucose."

tokens = token_pattern.findall(text)

print(tokens)
```

Output:

['Patient', 'has', '120.5', 'mg', '/', 'dL', 'glucose', '.']

The expression identifies dates, words, hyphenated terms, decimal numbers, and punctuation.

3.3 Maximal-Munch Tokenisation

Maximal-munch tokenisation selects the longest valid token at each position.

Input	Whitespace splitting	Improved tokenisation
500mg	500mg	500, mg
2026-11-05	2026-11-05	2026-11-05
Post-operative	Post-operative	Post-operative

Whitespace splitting may fail to separate numerical values and units correctly.

A specialised tokenizer can identify these components separately.

3.4 Normalisation Rules

The following normalisation rules are used:

1. Convert text to lowercase.
2. Expand common clinical abbreviations.
3. Standardise date formats.
4. Standardise measurement units.
5. Remove unnecessary punctuation.

Example:

```
abbreviations = {
    "bp": "blood pressure",
    "hr": "heart rate",
    "temp": "temperature",
    "htn": "hypertension",
    "dm": "diabetes mellitus",
    "od": "once daily",
```

```
    "bd": "twice daily"
}

text = "BP 120/80, HR 80"

text = text.lower()

for short, full in abbreviations.items():
    text = re.sub(
        rf"\b{short}\b",
        full,
        text
    )

print(text)
```

Output:

blood pressure 120/80, heart rate 80

3.5 Irreversible Normalisation

Some normalisation operations destroy information.

Examples include:

- Removing punctuation may destroy sentence boundaries.
- Converting all text to lowercase may remove distinctions between abbreviations.
- Removing units from measurements may destroy clinical meaning.

These operations are acceptable only when the lost information is not necessary for the intended search.

3.6 Limitations of Regular Expressions

Regular expressions cannot recognise every possible language structure.

For example, the requirement to match balanced nested parentheses is not a regular language.

A finite automaton cannot count an unlimited number of opening parentheses and ensure that the same number of closing parentheses appears.

Therefore, a more powerful formalism is required for such structures.

Conclusion: Regular expressions are effective for tokenisation and normalisation, but they cannot fully represent complex linguistic structures.

TASK 4 – Finite State Automata: Design, Non-Determinism and Equivalence

4.1 Definition

A finite automaton is formally represented as:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where:

- Q= Set of states
- Σ= Input alphabet

- δ= Transition function
- q_0= Initial state
- F= Set of final states

Finite automata are used to recognise strings belonging to a particular language.

4.2 Patient Identifier Automaton

Consider the identifier format:

P123456

The identifier begins with P, followed by six digits.

The states are:

q0 → q1 → q2 → q3 → q4 → q5 → q6 → q7

The transition sequence is:

State	Input	Next state
q0	P	q1
q1	Digit	q2
q2	Digit	q3
q3	Digit	q4
q4	Digit	q5
q5	Digit	q6
q6	Digit	q7

State q7 is the accepting state.

Any incorrect character or an incorrect number of digits causes rejection.

4.3 Deterministic and Non-Deterministic Automata

A deterministic finite automaton (DFA) has exactly one transition for each state and input symbol.

A non-deterministic finite automaton (NFA) may have multiple transitions for the same input symbol.

An NFA may also contain epsilon transitions, which allow movement between states without consuming an input symbol.

4.4 NFA Example

Consider a simplified patient identifier:

P123

The format allows an optional hyphen:

P-123

P123

An NFA can use an epsilon transition to skip the optional hyphen.

q0 --P--> q1

q1 --ε--> q2

q1 -----> q3

q2 --digit--> q3

q3 --digit--> q4

q4 --digit--> q5

The accepting state is q5.

The NFA accepts both forms.

4.5 Subset Construction

Subset construction converts an NFA into a DFA.

Each DFA state represents a set of NFA states.

For example:

$$A = \{q_0\}$$
$$B = \{q_1, q_2\}$$

The transitions are computed by finding all possible NFA states reachable from the current set.

This process continues until no new subsets are produced.

4.6 DFA Minimisation

DFA minimisation reduces the number of states without changing the recognised language.

Equivalent states are merged.

This reduces memory requirements and may improve processing efficiency.

4.7 Thompson's Construction

Thompson's construction converts a regular expression into an NFA.

The basic operations are:

- Symbol construction
- Concatenation
- Union
- Kleene star

For example:

ab

is converted into two transitions:

q0 --a--> q1 --b--> q2

4.8 Pumping Lemma

The pumping lemma states that every sufficiently long string in a regular language can be divided into three parts:

$$w = xyz$$

such that:

1. $| xy | \leq p$
2. $| y | > 0$
3. $xy^i z$belongs to the language for every $i \geq 0$

The language:

$$L = \{(^n)^n : n \geq 0\}$$

represents strings containing equal numbers of opening and closing parentheses.

This language is not regular because a finite automaton cannot remember an unlimited number of opening parentheses.

Conclusion: Finite automata provide a mathematical foundation for recognising regular patterns and validating structured clinical information.

TASK 5 – Inflectional Morphology

5.1 Introduction

Inflectional morphology studies changes in words that express grammatical information without changing their basic lexical meaning or word category.

A morpheme is the smallest meaningful unit of language.

A stem is the main part of a word to which affixes are attached.

An affix is a meaningful element added to a stem.

5.2 Inflectional Paradigms

Base form	Inflected form
Fracture	Fractures
Patient	Patients
Diagnose	Diagnosed
Treat	Treating
Injure	Injured

5.3 Inflectional Rules

Plural formation

patient → patients

The suffix -s is added to form the plural.

Past tense formation

treat → treated

The suffix -ed is added to form the past tense.

Present participle formation

treat → treating

The suffix -ing is added to form the present participle.

5.4 Orthographic Rules

E-deletion

diagnose → diagnosing

The final e is removed before adding -ing.

Consonant doubling

admit → admitted

The final consonant is doubled before adding -ed.

Y-to-I conversion

injury → injuries

The final y changes to i before adding -es.

Sibilant rule

diagnosis → diagnoses

The suffix -es is used for certain endings.

5.5 Irregular Forms

Some words do not follow regular inflectional rules.

Examples:

Base form	Irregular form

Be	Was
Have	Had
Go	Went
Child	Children
Foot	Feet

These forms must be stored in a lexicon because they cannot be generated reliably using regular suffix rules.

5.6 Importance in Clinical Search

A search for "fracture" should also retrieve records containing "fractures".

Similarly, searching for "treat" should identify related forms such as "treated" and "treating".

Inflectional analysis therefore improves search recall.

Conclusion: Inflectional morphology connects grammatical word forms to their underlying stems and improves clinical information retrieval.

TASK 6 – Derivational Morphology

6.1 Introduction

Derivational morphology studies how new words are formed by adding prefixes and suffixes.

Unlike inflection, derivation may change the meaning or grammatical category of a word.

6.2 Derivational Affixes

Affix	Meaning	Example
-itis	Inflammation	Arthritis
-ectomy	Surgical removal	Nephrectomy
-ology	Study of	Cardiology
-osis	Condition	Fibrosis
-emia	Blood condition	Anaemia
-ation	Process	Examination
-able	Capability	Treatable
-ive	Having a quality	Preventive
nephro-	Kidney	Nephrology
hyper-	Excessive	Hypertension
hypo-	Below normal	Hypoglycaemia
re-	Again	Reassessment
un-	Not	Unstable
-al	Related to	Renal
-ic	Related to	Cardiac

6.3 Derivational Families

Kidney-related words

Nephro

|

+-- Nephrology

|

+-- Nephrectomy

Heart-related words

Cardio

|

+-- Cardiology

|

+-- Cardiac

Treatment-related words

Treat

|

+-- Treatment

|

+-- Treatable

6.4 Semantic Non-Compositionality

The meaning of a derived word is not always predictable from its parts.

For example, the meaning of a medical term may depend on its established clinical usage rather than the literal meanings of its components.

A system that removes derivational suffixes aggressively may therefore produce misleading search results.

6.5 Derivational Conflation

Conflation means treating related words as equivalent during search.

For example:

Nephrectomy

Nephrectomies

These forms should be connected because they refer to the same procedure.

However, the following terms should not automatically be treated as equivalent:

Renal failure

Renal function

They share a root but have different clinical meanings.

Conclusion: Derivational morphology improves medical vocabulary analysis, but derivational relatives must be connected carefully to avoid misleading clinical search results.

TASK 7 – Finite-State Morphological Parsing

7.1 Introduction

A finite-state transducer (FST) is a computational model that maps one sequence of symbols to another.

In morphological analysis, it maps surface words to their lexical forms and grammatical features.

For example:

fractures

|

v

fracture + N + PL

7.2 Lexical and Surface Tapes

The lexical tape contains the underlying word and its grammatical features.

The surface tape contains the actual written word.

Example:

Lexical: fracture + N + PL

Surface: fractures

The transducer connects these two representations.

7.3 Morphological Parsing

The parsing process identifies the stem and features of a word.

Examples:

Surface form	Analysis
Fractures	fracture + N + PL
Patients	patient + N + PL
Treated	treat + V + PAST
Treating	treat + V + PROG
Diagnosed	diagnose + V + PAST

7.4 Continuation Classes

Continuation classes organise morphological resources according to the types of affixes that can follow a stem.

Examples:

- Noun stems may accept plural suffixes.
- Verb stems may accept tense suffixes.
- Adjective stems may accept comparative suffixes.

This organisation reduces the need to list every possible surface form separately.

7.5 Parsing and Generation

An FST can work in two directions.

Parsing:

fractures → fracture + N + PL

Generation:

fracture + N + PL → fractures

The same transducer supports both operations.

7.6 Ambiguity

A surface form may have multiple analyses.

For example:

treating

may be analysed as:

treat + V + PROG

or as part of a larger lexical structure.

The transducer may return multiple analyses.

A downstream component can use sentence context to choose the most appropriate interpretation.

Conclusion: Finite-state morphological parsing connects surface word forms to their lexical representations and supports both analysis and generation.

TASK 8 – Orthographic Rules and Two-Level Morphology

8.1 Introduction

Orthographic rules describe spelling changes that occur when morphemes are combined.

A lexicon transducer alone cannot always generate the correct surface spelling because English spelling rules modify the stem or affix at morphological boundaries.

8.2 Orthographic Rules

Rule 1: E-deletion

diagnose + ing → diagnosing

Rule 2: Consonant doubling

admit + ed → admitted

Rule 3: Y-to-I conversion

injury + es → injuries

Rule 4: Sibilant suffix

diagnosis + plural → diagnoses

Rule 5: E-deletion before certain suffixes

operate + ion → operation

8.3 Sequential Rule Cascade

The processing sequence is:

Lexicon

|

v

Morphological Analysis

|

v

Orthographic Rules

|

v

Surface Word

The lexicon identifies the stem and affixes.

The orthographic rules modify the spelling to produce the correct surface form.

8.4 Two-Level Morphology

Two-level morphology represents the relationship between lexical and surface forms directly.

Instead of applying spelling changes sequentially, the system evaluates lexical and surface symbols in parallel.

This provides a compact method for describing morphological spelling rules.

8.5 Advantages

A sequential cascade is easier to understand and implement.

A two-level system can represent multiple spelling relationships more systematically.

Both approaches reduce the need to store every inflected form separately.

Conclusion: Orthographic rules are necessary for generating correctly spelled clinical terms from their underlying morphological representations.

TASK 9 – The Porter Stemmer

9.1 Introduction

The Porter stemmer is a rule-based algorithm used to reduce English words to simplified stems.

It removes common suffixes through a sequence of predefined rules.

Unlike morphological lemmatisation, stemming does not require a complete dictionary of word forms.

9.2 Consonants and Vowels

In the Porter algorithm, vowels are:

a, e, i, o, u

The letter y may function as a vowel or consonant depending on its position.

The measure $m$represents the number of vowel-consonant sequences in a word.

For example:

trouble

The structure is:

CVCVVC

The measure is:

$$m = 2$$

9.3 Porter Stemming Steps

The Porter algorithm contains five major steps.

Step 1: Removes common inflectional endings.

Step 2: Replaces certain derivational suffixes.

Step 3: Removes additional derivational suffixes.

Step 4: Removes selected suffixes under specific conditions.

Step 5: Removes final e and reduces double l in certain cases.

9.4 Python Implementation

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

words = [
    "fractures",
    "fractured",
    "fracturing",
    "patients",
    "treated",
    "diagnoses"
```

]

```
for word in words:
    print(word, "->", stemmer.stem(word))
```

Output:

fractures -> fractur

fractured -> fractur

fracturing -> fractur

patients -> patient

treated -> treat

diagnoses -> diagnos

The stemmer removes suffixes without requiring a medical dictionary.

9.5 Stemming vs Lemmatisation

Feature	Stemming	Lemmatisation
Method	Rule-based suffix removal	Linguistic analysis
Dictionary	Not required	Usually required
Output	Simplified stem	Valid lexical form
Accuracy	Lower	Higher
Speed	Generally faster	May require more processing

For example:

fractures → fractur

is a possible Porter stem.

A morphological analyser produces:

fractures → fracture + N + PL

The second representation preserves more linguistic information.

9.6 Over-Stemming and Under-Stemming

Over-stemming occurs when unrelated words are reduced to the same stem.

Under-stemming occurs when related words are not reduced to the same stem.

Both errors can reduce the accuracy of clinical search.

Conclusion: The Porter stemmer is useful for reducing vocabulary variation, but a complete morphological analyser is more suitable for precise clinical retrieval.

TASK 10 – Implement and Test the Pipeline

10.1 Introduction

The pipeline combines regular expressions, tokenisation, normalisation, morphological analysis, and stemming.

The implementation uses Python's standard library and NLTK.

10.2 Python Code

```
import re
from nltk.stem import PorterStemmer
```

```
stemmer = PorterStemmer()

patterns = {
    "Patient ID": r"\bP\d{6}\b",
    "Date": r"\b\d{4}-\d{2}-\d{2}\b",
    "Blood Pressure": r"\b\d{2,3}/\d{2,3}\s*mmHg\b",
    "Age": r"\b\d{1,3}\s*years?\b"
}

token_pattern = re.compile(
    r"\d{4}-\d{2}-\d{2}|"
    r"[A-Za-z]+(?:[-'][A-Za-z]+)*|"
    r"\d+(?:\.\d+)?|"
    r"[^\w\s]"
)

abbreviations = {
    "bp": "blood pressure",
    "hr": "heart rate",
    "temp": "temperature"
}

def normalise(text):

    text = text.lower()

    for short, full in abbreviations.items():
        text = re.sub(
            rf"\b{short}\b",
            full,
            text
        )

    return text

def tokenize(text):
    return token_pattern.findall(text)

def stem_words(words):
    return [stemmer.stem(word) for word in words]
```

```
def extract_entities(text):

    results = {}

    for name, pattern in patterns.items():

        results[name] = re.findall(
            pattern,
            text,
            re.IGNORECASE
        )

    return results

text = """
Patient P123456, age 45 years.
Blood pressure 120/80 mmHg.
"""

normalised = normalise(text)

tokens = tokenize(normalised)

stems = stem_words(tokens)

entities = extract_entities(text)

print("Normalised Text:")
print(normalised)

print("\nTokens:")
print(tokens)

print("\nStems:")
print(stems)

print("\nEntities:")
print(entities)
```

10.3 Expected Output

```
Normalised Text:
patient p123456, age 45 years.
```

blood pressure 120/80 mmhg.

Tokens:

['patient', 'p123456', ',', 'age', '45',

'years', '.', 'blood', 'pressure', '120',

'/', '80', 'mmhg', '.']

The entity extraction stage identifies the patient identifier, age, and blood pressure.

10.4 Testing

The system should be tested using:

- Valid and invalid patient identifiers
- Different date formats
- Drug names with strengths
- Regular and irregular word forms
- Ambiguous medical terms
- Clinical abbreviations

Each test should record the expected output and the actual output.

10.5 Evaluation Metrics

Precision is calculated as:

$$Precision = \frac{TP}{TP + FP}$$

Recall is calculated as:

$$Recall = \frac{TP}{TP + FN}$$

F1-score is calculated as:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Where:

- TP = True positives
- FP = False positives
- FN = False negatives

These metrics evaluate the accuracy of the system.

Conclusion: The implementation demonstrates how the major components of the hospital search pipeline can be integrated using Python.

TASK 11 – Compare, Recommend and Defend the Normalisation Design

11.1 Comparison

Three normalisation strategies are considered.

Feature	No Normalisation	Porter Stemmer	Morphological Analysis
Processing	Direct text search	Suffix removal	Linguistic analysis

Recall	Lower	Higher	Potentially highest
Precision	May be high	May decrease	Can preserve distinctions
Lexicon	Not required	Not required	Required
Maintenance	Low	Low	High
Linguistic information	Limited	Reduced	Preserved

11.2 Recommended Design

The recommended design is a hybrid approach combining:

1. Regular expressions for structured entity recognition.
2. Finite-state automata for validation.
3. Morphological analysis for inflectional forms.
4. Controlled derivational analysis.
5. Porter stemming as a fallback for unknown words.

This approach balances search recall and precision.

11.3 Limitations

The system may fail when:

- Medical terms are misspelt.
- New terminology is introduced.
- Abbreviations have multiple meanings.
- The input contains non-English words.
- OCR introduces character errors.

The system also requires manual effort to build and maintain the lexicon.

Conclusion: A hybrid normalisation system is more suitable for hospital records because it combines efficient pattern recognition with linguistically informed analysis.

TASK 12 – Day-4 Design Challenge

Re-evaluate and Adapt the Design

A. Impact Analysis

The original system was designed for clean, typed English clinical records.

The new input contains OCR errors, handwritten text, Romanised regional language, and code-mixed sentences.

Example:

patient ku fever irundhuchu,

paracetamol 500mg kuduthen

The original system may fail to recognise some of these words because they are not included in its English lexicon.

OCR may also produce errors such as:

0 → O

1 → l

rn → m

These errors affect regular expression matching and morphological analysis.

B. Evaluation of the Original Design

Consider the following OCR example:

Patient P12345O admitted on 2026-11-05.

The patient identifier may be rejected because the last character is O instead of zero.

A code-mixed example:

patient ku fever irundhuchu

may be tokenised correctly, but the English morphological analyser may not identify the regional-language forms.

A combined example:

Patient P12345O ku fever irundhuchu.

may contain both an invalid identifier and unknown vocabulary.

The original system therefore requires additional mechanisms to handle noisy and multilingual input.

C. Adaptation or Redesign

The recommended approach is to modify the original pipeline rather than completely replace it.

The revised pipeline is:

OCR and Clinical Text

|

v

Character-Level Correction

|

v

Language Identification

|

v

Tokenisation

|

v

Entity Recognition

|

v

Morphological Analysis

|

v

Unknown-Word Handling

|

v

Search Index

The following changes are recommended:

1. Character-level correction

Use edit-distance matching to identify possible OCR errors.

For example:

P12345O → P123450

The correction must be validated before changing clinical information.

2. Expanded lexicon

Add common regional-language forms and their English equivalents.

3. Language identification

Identify whether each token belongs to English, a regional language, or another language.

4. Unknown-word handling

Use character-level similarity and fallback stemming for words that are not present in the lexicon.

5. Weighted automata

Use weighted transitions to represent alternative interpretations of noisy input.

D. Evidence-Based Defence

The modified system is recommended because the original system assumes clean English text.

The revised design provides:

- Better coverage of noisy input.
- Improved handling of OCR errors.
- Support for regional-language expressions.
- More flexible entity recognition.
- Better handling of unknown words.

However, the system still has limitations.

It cannot guarantee the correct interpretation of every misspelt or ambiguous medical term.

Clinical information must therefore be validated before it is used for diagnosis or treatment.

Final Conclusion

The proposed hospital record search system uses regular expressions, finite-state automata, morphological parsing, and stemming to improve clinical information retrieval.

The initial system provides an efficient foundation for structured and clean clinical text.

The modified system extends this foundation to handle OCR errors and code-mixed language.

A hybrid approach is recommended because it provides a balance between accuracy, efficiency, linguistic coverage, and maintainability.

The system can support clinicians by retrieving relevant records more effectively, but it should not replace clinical judgement.

MARK DISTRIBUTION

Task	Marks
Task 1 – Frame the Problem: Models, Algorithms and Levels of Analysis	3
Task 2 – Basic Regular Expression Patterns	4
Task 3 – Regular Expressions for Tokenisation and Normalisation	4
Task 4 – Finite State Automata: Design, Non-determinism and Equivalence	5
Task 5 – Inflectional Morphology	3
Task 6 – Derivational Morphology	4
Task 7 – Finite-State Morphological Parsing	5
Task 8 – Orthographic Rules and Two-Level Morphology	4
Task 9 – The Porter Stemmer	5
Task 10 – Implement and Test the Pipeline	3
Task 11 – Compare, Recommend and Defend the Normalisation Design	2
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**